Mobile Programming

FROM SUCCESSFUL API TO RELIABLE API — PRESIDENTKU

Membuat aplikasi mobile tetap berjalan dengan baik meskipun terjadi kegagalan pada request API.



PRESIDENTKU
Aplikasi mobile mengakses data dari server melalui API.

API

API REQUEST
Aplikasi mengirim request data ke API.

SUCCESS / ERROR
API mengembalikan response berhasil atau gagal.

HANDLE CONDITION
Aplikasi menangani setiap kondisi response (success, error, offline, timeout).

USER FEEDBACK
Aplikasi menampilkan informasi yang sesuai kepada pengguna.

Konsep Utama

Aplikasi yang baik harus tetap memberikan respons yang jelas kepada pengguna, meskipun API tidak dapat diakses.

Tidak semua request API selalu berhasil. Aplikasi perlu menangani kondisi berhasil, gagal, timeout, maupun tidak ada koneksi internet.

Pertanyaan Utama

Apa yang terjadi jika API tidak dapat diakses?

Mobile Programming

WHY DOES API REQUEST FAIL?

Memahami berbagai kemungkinan kegagalan ketika aplikasi melakukan melakukan request API.



MOBILE APP
Aplikasi PresidentKu melakukan request ke API.

API REQUEST
Aplikasi mengirim request data ke server melalui internet.

API

SUCCESS
Request berhasil dan data diterima.

HTTP ERROR
Server merespon dengan status error (4xx / 5xx).

TIMEOUT
Server terlalu lama merespon sehingga request dihentikan.

NO INTERNET
Tidak ada koneksi internet atau jaringan tidak tersedia.

1. SUCCESS
Request berhasil

Server dapat diakess dan mengembalikan data dengan status 2xx.

Contoh Response:

```
HTTP/1.1 200 OK
{
  "success": true,
  "data": [...]
}
```

2. HTTP ERROR
Server merespon error

Server dapat diakess, tetapi request menghasilkan status error seperti 4xx atau 5xx.

Contoh Response (401 Unauthorized):

```
HTTP/1.1 401 Unauthorized
{
  "success": false,
  "message": "Invalid token"
}
```

3. TIMEOUT
Server terlalu lama merespon

Server tidak memberikan respon dalam batas waktu tertentu.

Contoh Error:

```
TypeError: Network request timed out
(atau pesan timeout dari library)
```

4. NO INTERNET
Tidak ada koneksi internet

Perangkat tidak memiliki koneksi internet atau jaringan tidak tersedia.

Contoh Error:

```
TypeError: Failed to fetch
(atau pesan "Network request failed")
```

Key Takeaway

Tidak semua kegagalan API berarti aplikasi salah.
Kegagalan bisa disebabkan oleh error dari server, jaringan tidak tersedia, atau server yang terlalu lama merespon.

Fokus Pembahasan

1. **HTTP error (4xx / 5xx)** — Server merespons dengan status error
2. **Network error** — Tidak ada koneksi atau request gagal
3. **Timeout** — Server terlalu lama merespon
4. **Offline** — Tidak ada internet

Mobile Programming

SPRINT 05 — ERROR HANDLING & OFFLINE PRESIDENTKU

Membuat aplikasi PresidentKu tetap andal dengan menangani berbagai kondisi request API.



Alur Implementasi Sprint 05

API REQUEST
Aplikasi mengirim request data ke server

CHECK RESULT
Memeriksa hasil response dari server

OK
Request berhasil (2xx)

ERROR
Request gagal (4xx / 5xx)

OFFLINE
Tidak ada koneksi internet

DATA
Tampilkan data ke aplikasi

FEEDBACK
Tampilkan pesan error yang jelas

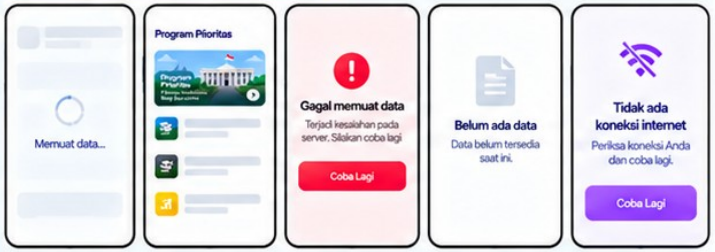
RETRY
Berikan opsi untuk mencoba lagi

Target Sprint 05

Mahasiswa akan mengimplementasikan:

1. Loading state
2. Success state
3. Error state
4. Empty state
5. Retry mechanism
6. Offline handling

Contoh Tampilan di Aplikasi



Key Takeaway

Aplikasi yang baik tidak hanya berhasil ketika API dapat diakses, tetapi juga memberikan feedback yang jelas ketika terjadi error, timeout, atau tidak ada koneksi internet.

Mobile Programming

HTTP STATUS CODE — WHAT HAPPENED TO THE REQUEST?

Kode status HTTP memberi tahu hasil dari request API. Response API tidak selalu berarti data berhasil diperoleh.

Kelompok Utama

2xx SUCCESS
Request berhasil dan server mengembalikan data.

4xx CLIENT ERROR
Kesalahan dari sisi client (misal: token salah, data tidak valid, resource tidak ditemukan).

5xx SERVER ERROR
Kesalahan dari sisi server (server error, service tidak tersedia).

Contoh HTTP Status Code

200	**OK** — Request berhasil.
400	**Bad Request** — Request tidak valid.
401	**Unauthorized** — Belum login atau token tidak valid.
403	**Forbidden** — Tidak memiliki izin akses.
404	**Not Found** — Data atau resource tidak ditemukan.
500	**Internal Server Error** — Terjadi kesalahan pada server.
503	**Service Unavailable** — Server tidak tersedia atau sedang maintenance.

Ilustrasi Response dari API

200 OK (Success)

```
HTTP/1.1 200 OK
{
  "success": true,
  "data": [...]
}
```

401 Unauthorized (Client Error)

```
HTTP/1.1 401 Unauthorized
{
  "success": false,
  "message": "Invalid token"
}
```

500 Internal Server Error

```
HTTP/1.1 500 Internal Server Error
{
  "success": false,
  "message": "Something went wrong"
}
```



Fokus Pembelajaran

Response API tidak selalu berarti data berhasil diperoleh.

Mahasiswa harus memeriksa status code dan menangani setiap kemungkinan response dengan tepat.

Mobile Programming

API UI STATES — EVERY REQUEST HAS A STATE

Setiap request API memiliki kondisi yang berbeda dan membutuhkan tampilan UI yang sesuai.

Alur UI State

IDLE
Belum melakukan request.

LOADING
Sedang mengirim request ke API.

SUCCESS
Request berhasil.

DATA
Tampilkan data dari API.

ERROR
Request gagal.

ERROR MESSAGE
Tampilkan pesan kesalahan.

EMPTY
Tidak ada data.

EMPTY STATE
Tampilkan pesan data kosong.



IDLE — Belum melakukan request.

LOADING — Sedang mengirim request ke API.

SUCCESS — Request berhasil dan data diterima.

ERROR — Request gagal dan tampilkan pesan kesalahan.

EMPTY — Request berhasil tetapi tidak ada data yang ditemukan.

Key Takeaway

UI tidak boleh hanya memiliki kondisi "data ada".
Setiap request API bisa menghasilkan loading, success, error, atau empty sehingga harus ditangani dengan UI yang sesuai.

State yang Perlu Diperhatikan

Loading
Menampilkan proses request yang sedang berjalan.

Success
Menampilkan data yang diterima dari API.

Error
Menampilkan pesan kesalahan jika request gagal.

Empty
Menampilkan pesan ketika tidak ada data.

Mobile Programming

TIMEOUT — WHAT IF THE SERVER TAKES TOO LONG?

Server tidak selalu merespons dengan cepat. Aplikasi perlu menangani timeout agar tidak terlihat hang.

Alur Timeout Handling

REQUEST
Aplikasi mengirim request ke API.

↓

WAITING
Menunggu response dari server.

↓

TIME LIMIT
Jika melebihi batas waktu yang ditentukan.

↓

TIMEOUT
Request dihentikan karena melebihi waktu.

↓

ERROR FEEDBACK
Menampilkan pesan error kepada pengguna.

Penjelasan

1. **Apa itu timeout?**
   Timeout adalah batas waktu maksimal untuk menunggu response dari server.

2. **Mengapa timeout diperlukan?**
   Agar aplikasi tidak menunggu terlalu lama ketika server lambat atau tidak merespons.

3. **Bagian dari network error**
   Timeout merupakan salah satu jenis network error (selain tidak ada internet atau koneksi gagal).

4. **Feedback kepada pengguna**
   Aplikasi perlu menampilkan pesan yang jelas dan memberikan opsi untuk mencoba lagi.

Contoh Tampilan di Aplikasi



Waiting / Loading
Aplikasi sedang menunggu response dari server.

Timeout Error
Menampilkan pesan timeout dan opsi untuk mencoba lagi.

Success (setelah retry)
Data berhasil dimuat setelah mencoba lagi.

Fokus Pembelajaran

Aplikasi tidak boleh terlihat "hang" ketika server terlalu lama merespons.
Mahasiswa harus mengimplementasikan timeout dan memberikan feedback yang jelas kepada pengguna.

Poin Penting

Tentukan batas waktu
Misal 10–30 detik untuk setiap request.

Tangani timeout
Sebagai bagian dari network error handling.

Berikan feedback jelas
Tampilkan pesan dan opsi coba lagi (retry).

Mobile Programming

OFFLINE HANDLING — WHAT IF THERE IS NO INTERNET?

Menangani kondisi ketika perangkat tidak memiliki koneksi internet agar aplikasi tetap memberikan feedback yang jelas.

Alur Offline Handling



Penjelasan

1. **Online**
   Perangkat memiliki koneksi internet dan dapat melakukan request ke API.
2. **Offline**
   Perangkat tidak memiliki koneksi internet (Wi-Fi atau data seluler mati).
3. **No internet**
   Tidak ada akses internet atau jaringan tidak tersedia.
4. **Network availability**
   Pemeriksaan ketersediaan jaringan sebelum melakukan request.
5. **Offline feedback**
   Tampilkan pesan yang jelas agar pengguna tahu apa yang harus dilakukan.

Contoh Tampilan di Aplikasi



Online (Normal)
Data berhasil dimuat dari API.

Offline State
Menampilkan pesan ketika tidak ada koneksi internet.

Checking Connection
Aplikasi memeriksa ketersediaan jaringan.

Key Takeaway

Aplikasi harus selalu memberikan feedback yang jelas ketika offline.
Jangan hanya menampilkan layar kosong. Berikan pesan yang mudah dipahami dan opsi untuk mencoba lagi.

Contoh Pesan UI

"Tidak ada koneksi internet.
Periksa koneksi Anda dan coba lagi."

Mobile Programming

CODE EXAMPLE — HANDLE API ERROR

Contoh implementasi kode untuk menangani response API yang berhasil maupun gagal.

Contoh Kode (JavaScript / React Native)

```js
const fetchData = async () => {
  try {
    const response = await fetch(url);

    if (!response.ok) {
      throw new Error("Request failed");
    }

    const data = await response.json();

    setData(data);
  } catch (error) {
    setError(error.message);
  }
};
```

Alur Proses



Contoh Hasil di Aplikasi



SUCCESS (Data)
Data berhasil dimuat dari API.

ERROR
Menampilkan pesan error ketika request gagal (response.ok = false).

LOADING
Menampilkan indikator loading saat request sedang diproses.

Key Takeaway

Selalu periksa response.ok sebelum memproses data.
Gunakan try-catch untuk menangani error dan tampilkan pesan yang jelas kepada pengguna jika terjadi kesalahan.

Hal yang Perlu Diperhatikan

1. Gunakan try-catch.
2. Periksa response.ok.
3. Tampilkan pesan error yang jelas.
4. Jangan menampilkan data jika request gagal.

Mobile Programming

TASK 01 — HANDLE API ERROR

Mahasiswa mengubah API integration PresidentKu agar mampu menangani berbagai kondisi response API.

Yang Harus Ditangani

2xx
Request berhasil dan data diterima.

4xx
Kesalahan dari sisi client (misal 400, 401, 403, 404).

5xx
Kesalahan dari sisi server (misal 500, 503).

Request gagal
Tidak ada koneksi internet atau network error.

Contoh Implementasi Kode

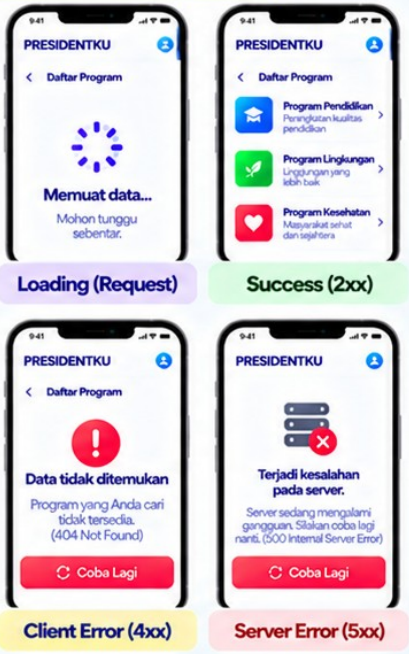
```js
const fetchPrograms = async () => {
  setLoading(true);
  setError(null);

  try {
    const response = await fetch(API_URL);

    if (!response.ok) {
      throw new Error('HTTP ${response.status}');
    }

    const data = await response.json();
    setData(data);
  } catch (error) {
    console.log('API Error:', error);
    setError(error.message || 'Terjadi kesalahan');
  } finally {
    setLoading(false);
  }
};
```

Contoh Hasil di Aplikasi



Checklist Implementasi

- [ ] **Response diperiksa** — Memeriksa response.ok dan status code.
- [ ] **HTTP error ditangani** — Menangani status 4xx dan 5xx.
- [ ] **Error state dibuat** — Membuat tampilan ketika terjadi kesalahan.
- [ ] **Error message ditampilkan** — Menampilkan pesan kesalahan yang jelas kepada pengguna.
- [ ] **Data tidak ditampilkan** — Tidak menampilkan data ketika request gagal.

Hasil yang Diharapkan

1. Aplikasi tidak crash ketika API gagal.
2. Pengguna mendapat pesan error yang jelas.
3. Data hanya ditampilkan ketika request berhasil (2xx).
4. Terdapat tombol coba lagi (retry).

Key Takeaway

Setiap request API harus selalu diperiksa dan ditangani semua kemungkinannya.
Dengan menangani 2xx, 4xx, 5xx, dan request gagal, aplikasi menjadi lebih stabil dan memberikan pengalaman pengguna yang lebih baik.

Mobile Programming

TASK 02 — LOADING, ERROR & RETRY

Mahasiswa mengimplementasikan loading indicator, error message, dan retry untuk request API pada PresidentKu.

Alur Implementasi

REQUEST
Aplikasi mengirim request ke API.

↓

LOADING
Menampilkan indikator loading.

↓

ERROR
Jika request gagal, tampilkan pesan error.

↓

RETRY
Pengguna menekan tombol coba lagi.

↓

REQUEST AGAIN
Aplikasi mengirim ulang request ke API.

Contoh Tampilan di Aplikasi



1. Request
Aplikasi melakukan request ke API.

2. Loading
Menampilkan loading indicator.

3. Error
Menampilkan pesan error yang jelas.

4. Retry
Melakukan request ulang ke API.

Yang Harus Dibuat Mahasiswa

1. **Loading indicator**
   Menampilkan proses request sedang berjalan.
2. **Error message**
   Menampilkan pesan ketika request gagal.
3. **Retry button**
   Tombol untuk melakukan request ulang.
4. **Request ulang**
   Aplikasi mengirim request kembali ketika tombol retry ditekan.

Checklist Implementasi

- [ ] **Loading indicator dibuat**
  Menampilkan indikator saat request.
- [ ] **Error message ditampilkan**
  Pesan error jelas dan mudah dipahami.
- [ ] **Retry button dibuat**
  Terdapat tombol untuk coba lagi.
- [ ] **Request ulang berhasil**
  Data dimuat kembali ketika retry.

Key Takeaway

Aplikasi harus memberikan feedback yang jelas pada setiap tahap request.
Dengan adanya loading, error, dan retry, aplikasi tidak terlihat seperti "hang" dan pengguna tahu apa yang sedang terjadi serta apa yang harus dilakukan.

Contoh Pesan Error



Mobile Programming

TASK 03 — HANDLE OFFLINE PRESIDENTKU

Mahasiswa mengimplementasikan penanganan kondisi offline agar aplikasi tetap memberikan feedback yang jelas dan dapat mencoba lagi (retry).

Alur Implementasi

REQUEST
Aplikasi melakukan request ke API.

↓

NO INTERNET
Tidak ada koneksi internet.

↓

OFFLINE STATE
Menampilkan tampilan offline.

↓

USER FEEDBACK
Memberikan pesan yang jelas kepada pengguna.

↓

RETRY
Pengguna menekan tombol coba lagi.

Contoh Tampilan di Aplikasi



1. Loading (Online)
Aplikasi melakukan request ke API.

2. Offline State
Menampilkan pesan ketika tidak ada koneksi internet.

3. Retry
Pengguna menekan tombol coba lagi.

4. Data Berhasil Dimuat
Data berhasil dimuat setelah koneksi internet aktif kembali.

Langkah Pengujian oleh Mahasiswa

1. **Internet aktif**
   Pastikan perangkat terhubung ke internet.
2. **Internet dimatikan**
   Matikan koneksi internet (Wi-Fi atau data seluler).
3. **Membuka halaman data**
   Buka halaman yang melakukan request ke API.
4. **Request gagal**
   Pastikan request gagal karena tidak ada koneksi internet.
5. **Ofline message muncul**
   Aplikasi menampilkan pesan offline yang jelas.
6. **Internet diaktifkan kembali**
   Aktifkan kembali koneksi internet.
7. **Retry**
   Tekan tombol coba lagi.
8. **Data berhasil dimuat**
   Pastikan data berhasil dimuat setelah retry.

Key Takeaway

Aplikasi harus selalu memberikan feedback yang jelas ketika offline.
Jangan menampilkan layar kosong. Berikan pesan yang mudah dipahami dan sediakan tombol coba lagi (retry) agar pengguna dapat mencoba ulang.

Contoh Pesan Offline

"Tidak ada koneksi internet.
Periksa koneksi Anda dan coba lagi."

Coba Lagi

Mobile Programming

DEMO — SHOW YOUR RELIABLE API

Mahasiswa menunjukkan aplikasi PresidentKu yang sudah mampu menangani loading, success, error, dan kondisi offline dengan baik.

Skenario 1 — Online (Success)

1. **RUN** — Jalankan aplikasi PresidentKu.
2. **API REQUEST** — Aplikasi melakukan request ke API.
3. **LOADING** — Menampilkan loading indicator.
4. **SUCCESS** — Data berhasil dimuat.



1. Run — Aplikasi dibuka.

2. Loading — Menampilkan indikator loading.

3. Success — Data program berhasil dimuat.

Skenario 2 — Offline (Error → Retry → Success)

1. **RUN** — Jalankan aplikasi.
2. **NO INTERNET** — Koneksi internet dimatikan.
3. **ERROR / OFFLINE** — Menampilkan pesan offline.
4. **RETRY** — Tekan tombol coba lagi.
5. **SUCCESS** — Data berhasil dimuat.



1. Run — Aplikasi dibuka.

2. Offline State — Menampilkan pesan offline.

3. Retry — Tekan tombol coba lagi.

4. Success — Data berhasil dimuat.

Pertanyaan Demo

1. Apa perbedaan HTTP error dan network error?
2. Apa fungsi loading state?
3. Apa yang terjadi ketika API mengembalikan 404?
4. Apa yang terjadi ketika server mengembalikan 500?
5. Apa yang terjadi ketika perangkat offline?
6. Bagaimana mekanisme retry bekerja?

Checklist Demo

- [ ] Menunjukkan loading state
- [ ] Menunjukkan data berhasil dimuat
- [ ] Menunjukkan error (HTTP error)
- [ ] Menunjukkan offline state (no internet)
- [ ] Menekan tombol retry
- [ ] Data berhasil dimuat setelah retry

Key Takeaway

Aplikasi yang baik harus tetap memberikan feedback yang jelas dalam semua kondisi.
Dengan menangani loading, success, error, dan offline, aplikasi menjadi lebih stabil., mudah digunakan, dan memberikan pengalaman pengguna yang lebih baik.

Contoh Pesan Offline

"Tidak ada koneksi internet.
Periksa koneksi Anda dan coba lagi."

Coba Lagi

Mobile Programming

REVIEW — FROM FRAGILE API TO RELIABLE API

Kita sudah belajar bagaimana menangani berbagai kondisi agar aplikasi PresidentKu menjadi lebih stabil, jelas, dan nyaman digunakan.

BEFORE — Fragile API

PresidentKu
Aplikasi

↓

API
Request ke server

↓

DATA
Data ditampilkan

Masalah:
Jika request gagal, aplikasi tidak memberikan feedback yang jelas. Pengguna hanya melihat layar kosong atau aplikasi terlihat "hang".



Request gagal
tidak ada pesan yang jelas.

Terlihat "hang"
ketika server lambat.

AFTER — Reliable API

PresidentKu
Aplikasi

↓

REQUEST
Mengirim request ke API

↓

LOADING
Menampilkan indikator loading

↓

CHECK RESULT
Memeriksa hasil response

DATA
Data berhasil dimuat

ERROR
Terjadi kesalahan (4xx / 5xx)

↓

RETRY
Request ulang

OFFLINE
Tidak ada koneksi internet

↓

RETRY
Request ulang



Success
Data berhasil dimuat.

HTTP Error (4xx / 5xx)
Pesan error yang jelas.

Offline
Pesan offline yang jelas.

Loading
Indikator loading.

Review Checklist

- **HTTP status code dipahami**
  Memahami arti 2xx, 4xx, 5xx.
- **HTTP error ditangani**
  Menangani response error (4xx, 5xx) dengan pesan yang jelas.
- **Network error ditangani**
  Menangani request gagal / tidak ada koneksi.
- **Loading state tersedia**
  Menampilkan indikator loading saat request.
- **Error state tersedia**
  Menampilkan pesan error ketika terjadi kesalahan.
- **Empty state tersedia**
  Menampilkan tampilan ketika data kosong.
- **Retry tersedia**
  Menambahkan tombol retry untuk request ulang.
- **Timeout ditangani**
  Menangani server yang terlalu lama merespons.
- **Offline condition ditangani**
  Menangani kondisi tidak ada koneksi internet.

Key Takeaway

Dengan menangani **loading, error, offline, retry**, dan berbagai kondisi lainnya, aplikasi **PresidentKu menjadi lebih stabil**, jelas, dan memberikan pengalaman pengguna yang lebih baik.

Tujuan Akhir

Membangun aplikasi yang tahan terhadap berbagai kondisi jaringan dan server, serta memberikan feedback yang jelas kepada pengguna.

Mobile Programming

SPRINT 05 — ERROR HANDLING & OFFLINE

Checklist akhir untuk memastikan aplikasi PresidentKu sudah mampu menangani berbagai kondisi sehingga lebih stabil, jelas, dan siap digunakan.

FINAL CHECKLIST

Pastikan semua poin sudah selesai sebelum melanjutkan ke sprint berikutnya.

Fitur yang Sudah Dipelajari

- **HTTP Status Code** — Memahami 2xx, 4xx, 5xx.
- **HTTP Error Handling** — Menangani response error dengan jelas.
- **Network Error Handling** — Menangani request gagal / tidak ada koneksi.
- **Loading State** — Menampilkan indikator loading saat request.
- **Success State** — Menampilkan data ketika request berhasil.
- **Error State** — Menampilkan pesan error yang jelas.
- **Empty State** — Menampilkan tampilan ketika data kosong.
- **Timeout Handling** — Menangani server yang terlalu lama merespons.
- **Retry Mechanism** — Menambahkan tombol retry untuk request ulang.
- **Offline Handling** — Menangani kondisi tidak ada koneksi internet.
- **Offline Feedback** — Menampilkan pesan offline yang jelas.

Contoh Tampilan di Aplikasi PresidentKu



Loading State — Menampilkan indikator loading saat request.

Success State — Data berhasil dimuat.

Error State — Menampilkan pesan error yang jelas dan tombol retry.



Empty State — Tampilan ketika data kosong.

Offline State — Menampilkan pesan offline dan tombol retry.

Timeout State — Menangani server yang terlalu lama merespons.

Final Checklist Sprint 05

1. ☐ **HTTP 2xx dipahami** — Memahami response sukses.
2. ☐ **HTTP 4xx dipahami** — Memahami client error.
3. ☐ **HTTP 5xx dipahami** — Memahami server error.
4. ☐ **HTTP error handling** — Menangani response error.
5. ☐ **Network error handling** — Menangani request gagal.
6. ☐ **Loading state** — Menampilkan indikator loading.
7. ☐ **Success state** — Menampilkan data sukses.
8. ☐ **Error state** — Menampilkan pesan error.
9. ☐ **Empty state** — Menampilkan tampilan kosong.
10. ☐ **Timeout handling** — Menangani request yang lama.
11. ☐ **Retry mechanism** — Menambahkan tombol retry.
12. ☐ **Offline handling** — Menangani kondisi offline.
13. ☐ **Offline feedback** — Menampilkan pesan offline.
14. ☐ **Retry setelah online** — Request ulang setelah online kembali.
15. ☐ **Test successful request** — Menguji request yang berhasil.
16. ☐ **Test HTTP error** — Menguji error 4xx dan 5xx.
17. ☐ **Test network error** — Menguji request gagal / tidak ada koneksi.
18. ☐ **Test timeout** — Menguji server yang lambat.
19. ☐ **Test offline** — Menguji kondisi offline.
20. ☐ **Test pada Expo Go** — Menguji aplikasi di Expo Go.
21. ☐ **Commit** — Melakukan commit perubahan.
22. ☐ **Push** — Melakukan push ke repository.
23. ☐ **Ready for next sprint** — Semua fitur berfungsi dengan baik.

Key Takeaway

Dengan mengimplementasikan error handling, timeout, dan offline handling, aplikasi PresidentKu menjadi lebih stabil, jelas, dan nyaman digunakan dalam berbagai kondisi jaringan.

Tujuan Sprint 05

Aplikasi PresidentKu mampu menangani berbagai kondisi error dan offline sehingga siap digunakan di dunia nyata dan siap untuk sprint selanjutnya.